PATTERNS - the FRED pattern matcher.

Summary of Patterns:

```
.          -- any single character (except new-line)
^          -- start of line
$          -- end of line
P*         -- zero or more of pattern P
P+         -- one or more of pattern P
P|Q        -- pattern P or Q
(P)        -- same as pattern P
[XYZ...]   -- any character inside brackets
[^XYZ...]  -- any character not inside brackets
{P}T       -- pattern P with tag T
<          -- beginning of word
>          -- end of word
@(N)       -- null string before column N
@(-N)      -- null string after Nth last column
@T         -- matches whatever matched last {P}
              with tag T
\E(name)   -- defined pattern
#          -- fence operation
```

Description:

Patterns are constructs that represent a set of character strings. A pattern *matches* a string if the string is in the set represented by the pattern. As a simple example,

```
/hello/
```

is a pattern that matches the string `hello`. Normally, FRED ignores whether characters are in upper or lower case, so the pattern also matches strings like `HELLO`, `Hello`, and `hELLo`. You can tell FRED to pay attention to the case of letters with the `O-SD` option.

Patterns need not be as simple as the one above. The pattern matching facility of FRED (called simply the *Pattern Matcher*) can handle very complicated patterns that match very diverse sets of strings. Because there are so many possible ways to specify a pattern, it is best to define the patterns accepted by FRED in a fairly rigorous manner. This is done below.

Note that we will use the S command to illustrate some of the patterns we describe.

```
s/pattern/string/
```

is a command puts the `string` in place of anything that matches `pattern`. Thus,

```
s/A/B/
```

changes every `A` in a line into a `B`.

- The simplest pattern is a single character. Such a pattern matches the given character in either upper or lower case, unless `O-SD` has been used to tell FRED to differentiate between cases.
- The character `^` matches the null string at the beginning of a line. Thus a command like

  ```
  s/^/A/
  ```

  would place an `A` at the beginning of a line.

- The character `$` matches the null string that occurs before the new-line character at the end of a line. In lines which do not have a nl on the end, `$` matches the physical end of the line. Thus

```
s/$/A/
```

will place an A at the end of a line.

- The character . matches any character except a new-line. Thus

```
s/./A/
```

will change every character in a line into an A (except for the nl on the end).

- Any pattern followed by a + matches a string of one or more occurrences of that pattern. Thus /B+/ matches the strings B, BB, BBB, etc. FRED will match a + construction with the longest sequence possible. For example, if a line contains the string BBBB, /B+/ will match all four B's as a unit, not individually.
- Any pattern followed by a * matches a string of zero or more occurrences of that pattern. Thus /B*/ matches the strings B, BB, BBB, etc. just as /B+/ does. In addition, since * patterns will match zero occurrences of a given pattern, /B*/ will match "" as well as longer strings of B's. As with +, the Pattern Matcher will match a * construction with the longest sequence possible.
- Up to now, we have been speaking of very simple patterns. More complicated patterns can be constructed by arranging a number of simpler patterns adjacently. For example,

  `/ab/`
  : matches any string "ab"

  `/..a/`
  : matches any two characters followed by "a"

  `/^a/`
  : matches line beginning with "a"

  `/ab*/`
  : matches "a", "ab", "abb", "abbb", ...

  `/.$/`
  : matches last character on line
- @(N) is a pattern that matches the null string immediately before the Nth column of a line. Thus if a line contains ABC, /@(2)/ will match the position between the A and the B.

  @(-N) is a pattern that matches the null string after the Nth column from the end of a line. Thus @(-1) matches the null string immediately before the nl character that ends the line. By convention, @(0) matches the null string immediately after the new-line character at the end of the line.

  @(N-) is a pattern that matches the null string before the Nth column in the line or before any previous characters. @(N+) is a pattern that matches the null string before the Nth column in the line or before any column later in the line (it will not match the null string after a nl). For example,

```
/A@(4-)/
```

  is a pattern that will match an A occurring as one of the first three characters in a line.

- Two patterns separated by | form a pattern that matches either pattern. Thus

```
/A|B/
```

  matches either A or B.

- Putting a pattern in parentheses creates a new pattern that matches the same strings as the original. However, since a pattern in parentheses is considered to be a single entity, the parentheses can be used to affect the order of evaluation of +, *, and |. Thus

```
/(AB)+/
```

  matches AB, ABAB, ABABAB, etc. whereas

```
/AB+/
```

matches AB, ABB, ABBB, etc. Similarly

```
/A(B|C)D/
```

will match either ABD or ACD while

```
/AB|CD/
```

will match AB or CD.

- /[string]/ is a pattern that matches any single character in string. Thus

```
/[1234567890]/
```

matches any single digit. Characters inside the square brackets are taken literally, without their special meanings.

```
/[.]/
```

matches a period, not "any character". Patterns of this form are equivalent to patterns of single characters joined by |. For example, the following are equivalent:

```
/[abcd]/
/a|b|c|d/
```

The string inside square brackets can contain constructs of the form

```
c1-c2
```

where c1 and c2 are ASCII characters. This stands for the *range* of ASCII characters from c1 to c2 (inclusive). For example,

```
[a-z]
```

matches all letters (upper or lowercase), provided that O+SD is in effect (see "expl fred os" for more on O+SD).

```
[a-z0-9]
```

matches all letters and digits. If you do not want both upper and lowercase letters, put a \C in front of the first letter in the range. For example,

```
[\Ca-z]
```

stands for the lowercase letters only. Similarly,

```
[\Oa-z]
```

stands for both upper and lowercase characters, regardless of options. Note that the \O or \C only goes in front of the first character of the range; it is an error to put it in front of the last character.

For dual case matching, both ends of the range must be alphabetic. For example, if you specify

```
[@-`]
```

(which includes the uppercase letters in its range but not the lower case ones), you only get single case matching. If you try

```
[a-`] "Error when O+SD
```

you get an error when O+SD is in effect; if one end of the range is a letter, the other must be too.

You may specify the ends of a range in either order; for example, the following are equivalent

```
[z-a] [a-z]
```

If you want a square bracket construct that matches the minus sign `'-'` as well as other characters, specify the character in a place that cannot be mistaken for a range. For example,

```
[-a-z]
```

matches the minus sign and all letters. You can also put a `\c` in front of the minus sign. The character `']'` can be used as the last character of a range without being mistaken for the end of square bracket construct. For example,

```
[[-]]
```

matches all the ASCII characters from `'['` to `']'`.

- `/[^string]/` is a pattern that matches any character **except** the characters of `string` and the new-line character. Thus

  ```
  /[^1234567890]/
  ```

  will match any non-nl character that is not a digit. Again, characters of "string" are taken literally so that

  ```
  /[^.]/
  ```

  matches any character that is not a period. Ranges can be used in `string` in the same way as `[string]`. For example,

  ```
  [^-a-z]
  ```

  matches everything except the minus sign and the letters.

- `/{pattern}t/` is a pattern in which `pattern` is "tagged" with the single character `t`. The tag `t` can be any character. If the pattern of an `S` command contains a tag, the tag can be used in the second part of the `S` command to stand for whatever the bracketed pattern matched. For example, in

  ```
  s/ME{OW}A/BA-WA/
  ```

  the tag `A` stands for the string `OW`. Thus the above command is effectively

  ```
  s/MEOW/BOW-WOW/
  ```

  Similarly,

  ```
  s/{A|B}X/XX/
  ```

  changes the string `ABCD` into `AABBCD` since each `A` or `B` is doubled by the substitution.

- The construct `@c` (where `c` is any alphabetic character) matches the string that was matched by a preceding `{pattern}c` construct in the same pattern. For example, in

  ```
  /{A*}x @x/
  ```

  the `@x` pattern matches exactly the same string matched by `{A*}x`. As a result, the above pattern matches zero or more `A` characters, followed by a space, followed by exactly the same string of `A` characters (including the case of the letters).

- Putting a `<` at the beginning of a pattern (e.g. `/<ABC/`) creates a new pattern which matches the same strings as the original, provided that these matching strings are immediately preceded by a space, tab, new-line, or non-alphanumeric character. The easiest way to think of this is that `/<ABC/` matches `ABC` when it occurs at the beginning of a word, but not when it occurs in the middle of a word. Thus `/<ABC/` would match the `ABC` in

```
abcdefghijk...
```

but it would not match the `ABC` in `DABC` because the `ABC` does not occur at the beginning of the word.

- Putting a `>` at the end of a pattern (e.g. `/ABC>/`) creates a new pattern which matches the same strings as the original, provided that these matching strings are immediately followed by a space, tab, new-line, or non-alphanumeric character. The easiest way to think of this is that `/ABC>/` matches `ABC` when it occurs at the end of a word, but not when it occurs in the middle of a word. Thus `/ABC>/` would match the `ABC` in

  ```
  nnnabc
  ```

  but it would not match the `ABC` in `ABCD` because the `ABC` is not at the end of the word.

  Note that `/<string>/` matches the complete word `string` but does not match `string` when it is part of another word. For example, `/<A>/` matches the word `A` in

  ```
  A CAT
  ```

  but not the `A` in `CAT` because that `A` is part of a larger word.

- If `patname` is the name of a pattern as defined in

  ```
  E(patname)/pattern/
  ```

  command, then `\E(patname)` is a pattern that matches the same strings as the original `pattern`. Recursive definitions of patterns in an `E` command are permitted. For example,

  ```
  E(bal)/[^()]*|\C(\E(bal)\C)/
  ```

  defines a pattern `(bal)` which matches any string of characters that does not contain parentheses, as well as any string in which parentheses are properly balanced. The `\E` character has no special meaning outside of patterns. If a pattern contains

  ```
  \E(name)
  ```

  where `name` is not the name of a named pattern, the construct terminates the matching attempt in the same way that `#` does (see below).

- `#` may be used in patterns in a way similar to the *fence* operation in SNOBOL. `#` prevents the Pattern Matcher from "backing up" and making another attempt to find a given pattern if the first attempt fails. For example, consider the pattern `/A#B/`. In its attempts to match this pattern, the Pattern Matcher moves across a line column by column, searching first for the letter `A`. When it finds an `A`, it looks for a `B` immediately after. If it does not find a `B`, it does not "back up" and continue its search for `AB` in the line. Instead, it stops scanning the line and says no match was found. Thus `/A#B/` will not match the line `AAB`, since the Pattern Matcher will not look any farther once it finds that the first `A` is not followed by a `B`.

  One use of this construction is in a command like

  ```
  s/^%#HE|THIS/THAT/
  ```

  If this command finds a line beginning with `%`, it checks whether the `%` is immediately followed by `HE`. If so, FRED changes the construct to `THAT` and goes on to scan the rest of the line. If not, FRED makes no attempt to scan the rest of the line. If the line does not begin with `%`, the command will scan the rest of the line and change any occurrence of `THIS` to `THAT`. The effect of the command therefore is to change `%HE` to `THAT`, and to change `THIS` to `THAT` on lines that don't begin with `%`.

  Concatenating similar constructions with or-bars `|` creates a pattern that excludes a number of conditions before substitutions are made. For example,

  ```
  s/(^%#$.)|(^.*:$#.)|A/B/
  ```

will change `A` to `B` on lines that neither begin with `%` nor end with `:`. You can get the same effect with

```
s/(^%\e(name))|(^.*:$\e(name))|A/B/
```

if there is no pattern named `(name)`.

- A null pattern `//` is equivalent to the pattern most recently encountered by FRED. This feature is convenient when searching through a file for a particular string. For example, if you are looking for a particular occurrence of `HELLO`, you can perform this search by typing `/HELLO/` the first time and typing `//` on subsequent searches. In this way, you can look through the file for each occurrence of `HELLO` without being forced to type the word every time.
- The null string is acceptable as a part of a pattern. The null string should not be confused with the null pattern `//`, since the null string cannot stand on its own. The null string can be used in a pattern such as

  ```
  /ABC(D|)/
  ```

  This pattern matches either `ABCD` or `ABC` and is somewhat shorter to type than `/ABCD|ABC/`. As another example,

  ```
  s/()/ /p
  ```

  puts a space between every character on the line.

Quasi-Patterns

If N is a positive integer,

```
N/pattern/
```

defines a *quasi-pattern* which matches the `N`th occurrence of `pattern` in any line. Thus you can say such things as

```
s2/a/x/ zu3/y/ zl1/v/
```

and so on. In the same way,

```
-N/pattern/
```

is a quasi-pattern that matches the `N`th occurrence of `pattern` from the **end** of any line.

We call these quasi-patterns instead of genuine patterns because these constructions are not valid in line addresses or in G and T commands (where they would be ambiguous). Otherwise, quasi-patterns can be used anywhere that a normal pattern can.

The constructions described above are the only patterns accepted by FRED. No other patterns are valid. No pattern will match strings that spread across more than one line.

Activating Patterns

Some of the constructions defined above can be activated and deactivated using the `O+S` or `O-S` commands. The default options are

```
O+S^$.*[\E&D-
O-S{(|+#@
```

Placing `\C` in front of one of the special pattern characters tells FRED to take the character literally, ignoring any special meaning. Placing `\O` in front of a pattern character tells FRED to use the character's special meaning, even if that meaning has been disabled by option commands. For example, if `O+S*` is in effect

```
s/A\C*/X/
```

will change the string `A*` to the letter `X`. This is far different from

```
s/a*/x/
```

In the same way, even if `O-S|` is in effect

```
s/A\O|B/C/
```

will change `A` or `B` into `C`.

Pattern Delimiters

Throughout this discussion of patterns we have used the slash `/` to delimit our patterns. While this is the general practice, FRED will accept any other non-alphabetic non-numeric character as a delimiter for patterns in `S` commands, `T` commands, and so on. However, patterns acting as line addresses can only be delimited by `/` and `?`.

Pattern Examples

`/ABCD/`
: matches `ABCD` anywhere in the line.

`/A(B|C)+D/`
: matches a string beginning with `A`, ending with `D` and having a number of `B`'s and/or `C`'s in between.

`/^BEGIN.*END$/`
: matches any line beginning with `BEGIN` and ending with `END`.

`/A[1234567890]/`
: matches `A` followed by a digit.

The Pattern Matcher's Search Process

The Pattern Matcher searches for a string to match a given pattern by moving across a line column by column. In other words, it usually checks to see if there is a suitable string beginning in column 1, then beginning in column 2, and so forth. The exception is when it is searching for a quasi-pattern with a negative qualifier as in `-1/A/`. In this case, FRED begins looking for the pattern column by column from the end of the line instead of the beginning.

In general, FRED looks for the longest suitable string beginning in a given column. One exception to this rule occurs sometimes in patterns like

```
/ABC|AB|A/
```

which use the or-bar `|`. For any given column, FRED will first search for a string matching `ABC`, and will only search for `AB` and `A` if it does not find `ABC`. The pattern

```
/AB|A|ABC/
```

will look for `AB` beginning in a given column before it looks for `A` and `ABC`. Since FRED will always find `A` before it finds the full string `ABC`, the above will match `AB` or `A`, but never `ABC`.

A few more examples are given below.

`/A|AB|ABC/`
: In a string `ABCD`, this matches `A`. Because FRED finds the match for `A` before `AB` or `ABC`, the `AB` and `ABC` are essentially useless in the pattern.

`/ABC|AB|A/`
: In a string `ABCD`, this matches `ABC`.

`/ABC|AB|A/`
: In a string `AABCD`, this will first match the first `A` and then the `ABC`.

In this way, FRED allows you to dictate which strings you prefer to match first.

A side effect of this principle lets you tell FRED to find the **shortest** matching string instead of the longest. A pattern like

```
/A.*B/
```

matches the longest string beginning in `A` and ending in `B`. If, however, you try

```
/A(|.)*B/
```

you tell FRED that you would rather match the null string (before the or-bar) than an actual character. Thus FRED will be satisfied with the shortest match rather than the longest (and you will match the shortest string beginning with an `A` and a `B`). If you have a line

```
ABAB
```

the command

```
s/A.*B/X/
```

will leave a line that only contains `X`, while

```
s/A(|.)*B/X/
```

will leave a line that contains `XX` (since both `AB` pairs turn into `X`).

Another consequence of FRED's approach to pattern matching is the way in which it handles a command like

```
s-1/A.*/X/
```

FRED moves column by column backwards across the line and matches `/A.*/` with the first `A` it comes to. Thus the given command will change only the last `A` in the line and everything after it.

As a last example, consider the pattern

```
/{.*}a{.*}b/
```

In this case, the pattern tagged with `A` matches the entire contents of the line, and the pattern tagged with `B` matches the null string.